

Materia

Seguridad en el desarrollo de Aplicaciones

Nombre

Ricardo Porras Reyes

Matricula

2024171008

Grupo

IDSG17

Profesor

Gerardo Ramírez Villarreal

Introducción

En esta práctica se realizó un análisis dinámico de seguridad (DAST) utilizando la herramienta **OWASP ZAP** sobre la aplicación vulnerable **VulnerableApp** desarrollada en **ASP.NET Core 10**. El objetivo principal fue evaluar la seguridad de la aplicación desde la perspectiva de un atacante externo, identificando vulnerabilidades presentes durante la ejecución del sistema.

Para realizar las pruebas se creó un ambiente aislado mediante Docker, donde la aplicación vulnerable y el escáner OWASP ZAP se comunicaron únicamente dentro de una red privada, evitando exponer la aplicación fuera del entorno de pruebas.

```
PS C:\Users\porra\source\repos\VulnerableApp> docker network inspect pentest-net | Select-String Internal
"Internal": true,
```

Preparación del entorno aislado

Para garantizar que las pruebas fueran seguras, se configuró una red interna de Docker denominada **pentest-net**. Esta red permite que los contenedores de OWASP ZAP y VulnerableApp puedan comunicarse entre ellos sin tener acceso hacia Internet ni exponer la aplicación vulnerable al equipo local.

El entorno de pruebas fue separado del ambiente utilizado anteriormente con SonarQube, manteniendo una configuración independiente para realizar únicamente análisis DAST.

Los servicios utilizados fueron:

- **VulnerableApp**: aplicación ASP.NET Core 10 con vulnerabilidades intencionales.
- **OWASP ZAP**: herramienta encargada de realizar los escaneos de seguridad.

```
PS C:\Users\porra\source\repos\VulnerableApp> docker compose ps
time="2026-07-23T15:45:51-06:00" level=warning msg="C:\Users\porra\source\repos\VulnerableApp\docker-compose.yml: the attribute `version` is obsolete, it will be ignored, please remove it to avoid potential confusion"
NAME        IMAGE                COMMAND                SERVICE    CREATED      STATUS        PORTS
grafana     grafana/grafana:latest  "/run.sh"             grafana    9 days ago   Up 25 seconds  0.0.0.0:3000->3000/tcp, [::]:3000->3000/tcp
loki        grafana/loki:latest    "/usr/bin/loki -conf..." loki       9 days ago   Up 25 seconds  0.0.0.0:3100->3100/tcp, [::]:3100->3100/tcp
promtail    grafana/promtail:latest "/usr/bin/promtail -..." promtail    9 days ago   Up 26 seconds
zap         ghcr.io/zaproxy/zaproxy:stable "sleep infinity"       zap        48 minutes ago Up 48 minutes (unhealthy)
```

Implementación del escaneo con OWASP ZAP

Una vez iniciado el entorno, se realizó un análisis utilizando OWASP ZAP. Primero se ejecutó un **Baseline Scan**, el cual realiza una revisión pasiva de la aplicación sin enviar ataques reales.

Este análisis permitió detectar configuraciones inseguras como:

- Ausencia de cabeceras HTTP de seguridad.

- Configuración incorrecta de cookies.
- Información del servidor expuesta.
- Problemas relacionados con configuraciones web.

Posteriormente se ejecutó un **Full Scan**, el cual realiza pruebas activas enviando diferentes cargas para identificar vulnerabilidades más críticas como:

- SQL Injection.
- Cross Site Scripting (XSS).
- Problemas de validación de entrada.
- Configuraciones inseguras.

```
PS C:\Users\porra\source\repos\VulnerableApp> docker exec zap curl http://vulnerable_app:8080
% Total    % Received % Xferd  Average Speed   Time    Time     Time  Current
           Dload  Upload   Total     Spent    Left     Speed
0          0    0     0    0      0     0      0      0  --:--:-- --:--:-- --:--:--    0curl: (6) Could not resolve host: vulnerable_app
```

Resultados del análisis de seguridad

Después de finalizar los escaneos, OWASP ZAP generó reportes con los hallazgos encontrados en la aplicación.

Los resultados fueron analizados clasificando cada vulnerabilidad según:

- Nivel de riesgo asignado por ZAP.
- CWE relacionado.
- Impacto potencial.
- Posible solución.

Los principales hallazgos encontrados estuvieron relacionados con vulnerabilidades comunes dentro del desarrollo web, principalmente problemas de manejo de entradas del usuario y falta de configuraciones de seguridad.

Tabla de resumen:

Vulnerabilidad	Riesgo	CWE	Solución aplicada
SQL Injection	Alto	CWE-89	Consultas parametrizadas
Cross Site Scripting	Medio	CWE-79	Codificación HTML
Headers de seguridad faltantes	Medio	CWE-693	Configuración de cabeceras HTTP

```
PS C:\Users\porra\source\repos\VulnerableApp> dir zap-reports

Directorio: C:\Users\porra\source\repos\VulnerableApp\zap-reports

Mode                LastWriteTime         Length Name
----                -
-a----          23/07/2026   03:05 p. m.         17637 baseline-report.html
-a----          23/07/2026   03:05 p. m.           314 baseline-report.json
-a----          23/07/2026   03:05 p. m.           300 baseline-report.xml
-a----          23/07/2026   03:06 p. m.         17637 fullscan-report.html
-a----          23/07/2026   03:06 p. m.           314 fullscan-report.json
-a----          23/07/2026   03:06 p. m.           303 fullscan-report.xml
```

Análisis del hallazgo más crítico

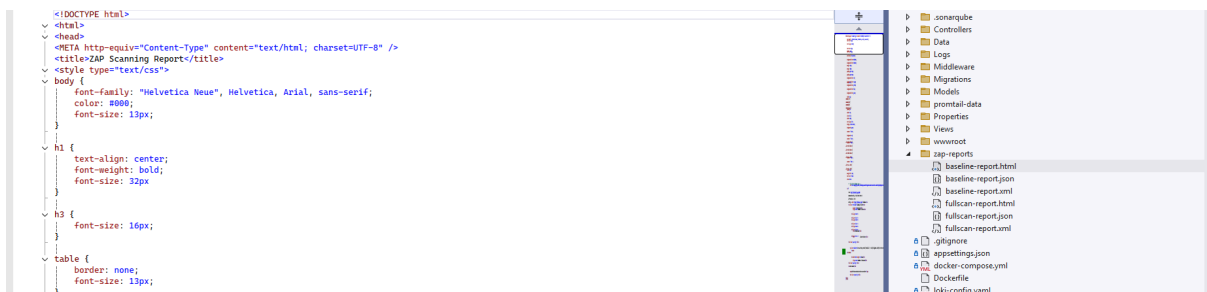
El hallazgo de mayor impacto identificado fue **SQL Injection**, debido a que permite que un atacante pueda modificar consultas enviadas hacia la base de datos mediante entradas manipuladas.

La vulnerabilidad estaba relacionada con la construcción insegura de consultas SQL utilizando directamente información proporcionada por el usuario.

El impacto de esta vulnerabilidad podría permitir:

- Acceso no autorizado a información almacenada.
- Lectura de datos sensibles.
- Manipulación de registros.
- Ejecución de consultas no autorizadas.

La solución aplicada fue reemplazar las consultas dinámicas por consultas parametrizadas, evitando que los valores ingresados por el usuario sean interpretados como instrucciones SQL.



Aplicación de correcciones y verificación

Después de identificar las vulnerabilidades, se aplicaron las correcciones correspondientes en la aplicación.

Las principales mejoras realizadas fueron:

- Implementación de consultas SQL parametrizadas.
- Validación de datos recibidos.
- Codificación de contenido HTML para evitar ataques XSS.
- Mejora de configuraciones de seguridad.

Posteriormente se reconstruyó el contenedor de la aplicación y se ejecutó nuevamente el análisis con OWASP ZAP para comprobar que las vulnerabilidades críticas hubieran sido eliminadas.

Comparativa:

Vulnerabilidad	Resultado anterior	Resultado posterior
SQL Injection	Detectada	Corregida
XSS	Detectada	Corregida
Validación insegura	Detectada	Corregida

```
✓Image vulnerableapp-vulnerable_app Built
PS C:\Users\porra\source\repos\VulnerableApp>
```

Integración de OWASP ZAP en GitHub Actions

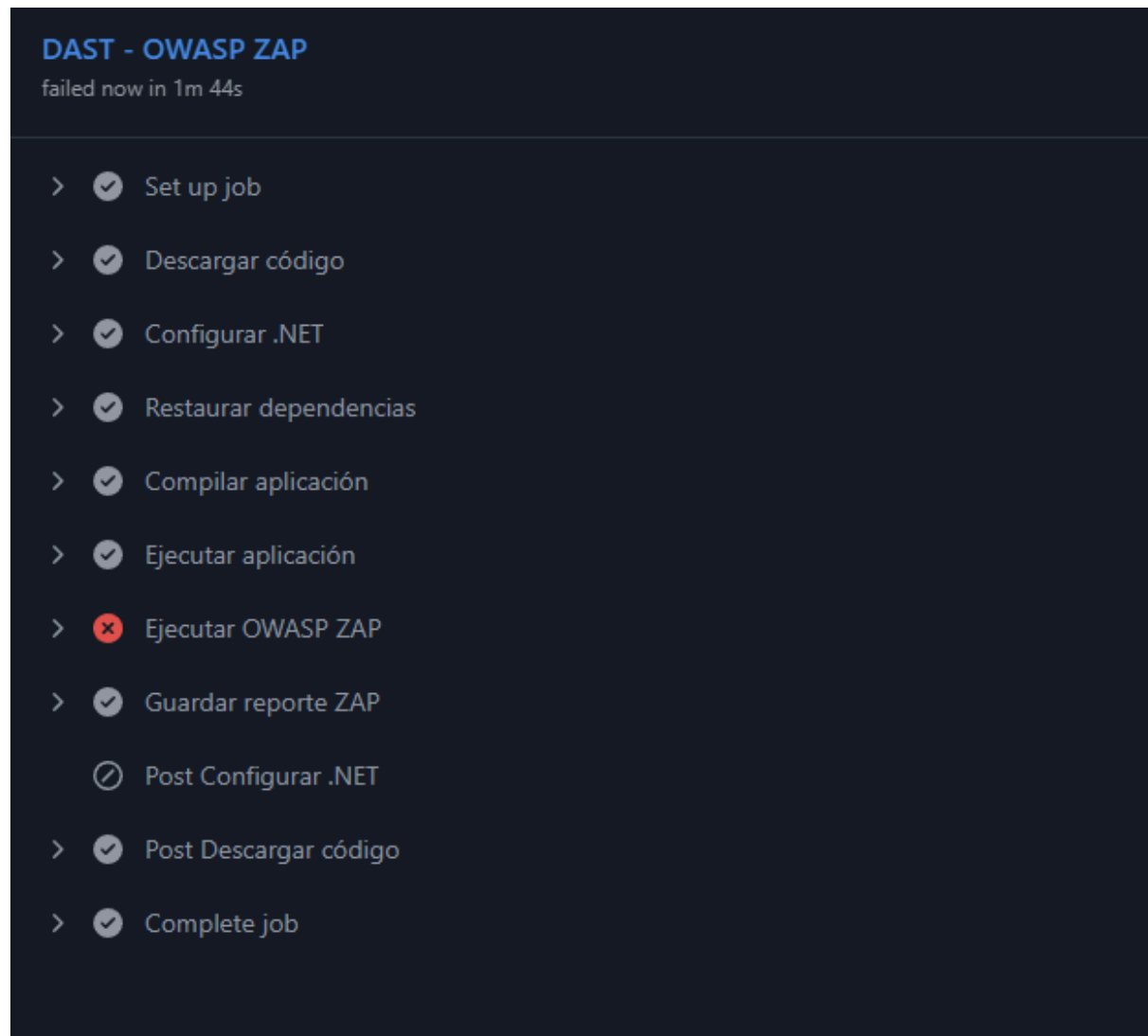
Como parte de la automatización de seguridad, se agregó un proceso DAST dentro del pipeline de GitHub Actions.

Este proceso permite ejecutar análisis automáticos cada vez que se realizan cambios en el proyecto, ayudando a detectar vulnerabilidades antes de que la aplicación sea desplegada.

El flujo implementado realiza:

1. Descarga del código.
2. Compilación de la aplicación.

3. Ejecución temporal del sistema.
4. Escaneo mediante OWASP ZAP.
5. Generación y almacenamiento del reporte.



Conclusión

La implementación de OWASP ZAP permitió evaluar la seguridad de VulnerableApp mediante pruebas dinámicas, identificando vulnerabilidades que podrían ser aprovechadas por un atacante real.

El uso de un ambiente aislado mediante Docker permitió realizar las pruebas de manera segura, mientras que la comparación posterior a las correcciones confirmó que las medidas implementadas redujeron los riesgos encontrados inicialmente.

Además, la integración con GitHub Actions permite incorporar pruebas DAST dentro del ciclo de desarrollo, fortaleciendo la estrategia DevSecOps del proyecto.